LINGUAGEM DE PROGRAMAÇÃO

Relacionamento

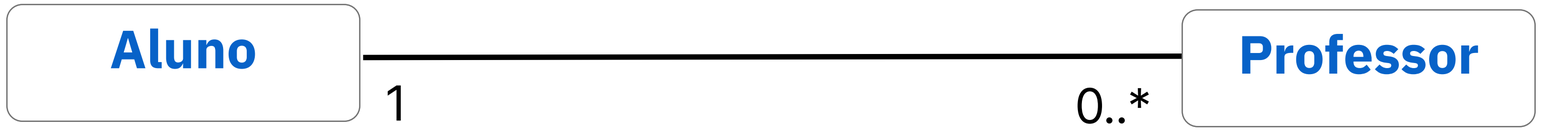
Relacionamento

- É a forma como classes e objetos **se conectam e iteragem** entre si para modelar situações do mundo real.

- Há 4 tipos de relacionamentos: associação, composição, agregação e herança.

Associação

- A associação indica que as classes **se relacionam de alguma forma** (seus objetos se relacionam em tempo de execução).
- No Diagrama de Classes da UML, a Associação é indicada por uma linha que une as 2 as classes.
- Definimos a **multiplicidade**, que representa quantos objetos de uma classe podem estar relacionados com objetos de outra classe dentro de um relacionamento.



Composição / Agregação

- Tipos de associação que indicam uma relação de **objeto todo e objeto parte**.
- Relacionamentos do tipo “**TEM-UM**” .

- Ex: Pedido “tem um” ItemPedido
- Ex: Equipe “tem um” Jogador

Padaria
Dona Maria

Número: 0001

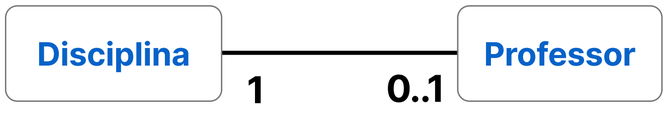
Data: 12/12/2012

Produto	Quantidade	Preço
Café	1	R$ 10,00
Pão na chapa	2	R$ 20,00

Total: R$ 30,00

Objeto como atributo

- Para implementar a composição ou agregação de classes, um objeto (todo) contém objetos de outras classes (partes).




```java
public class Disciplina {

  private String sigla;
  private int codigo;
  private Professor prof;
}
```

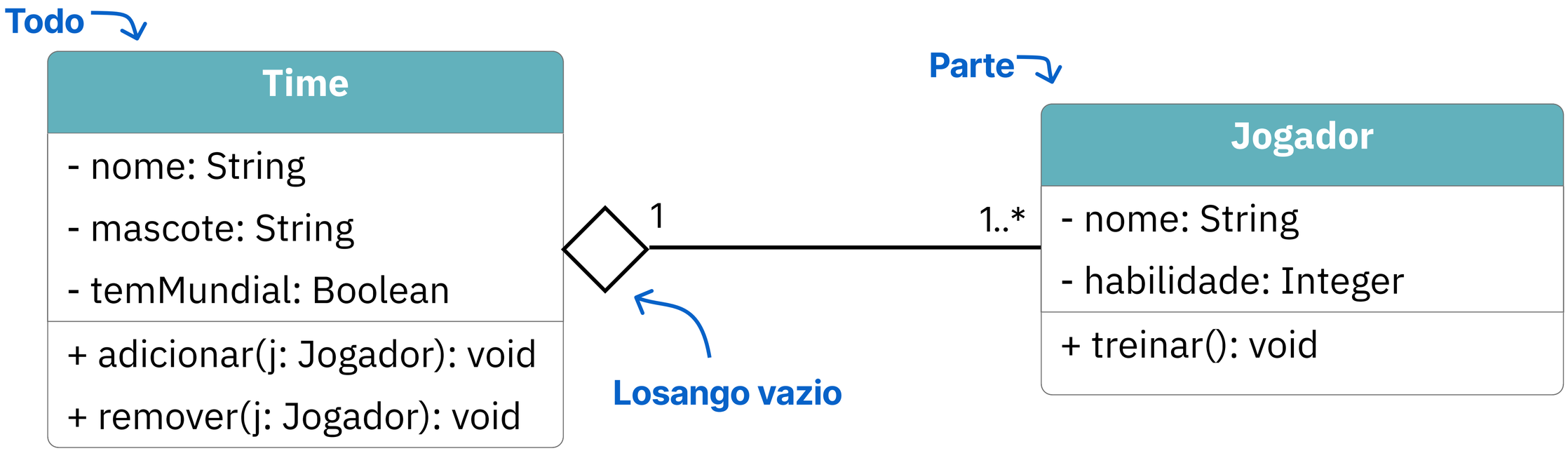



```java
public class Disciplina {

  private String sigla;
  private int codigo;
  private List<Professor> professores;
}
```

Agregação

- Objeto parte existe independente do objeto todo.
- Objeto parte pode eventualmente ser parte de outro objeto.



Agregação

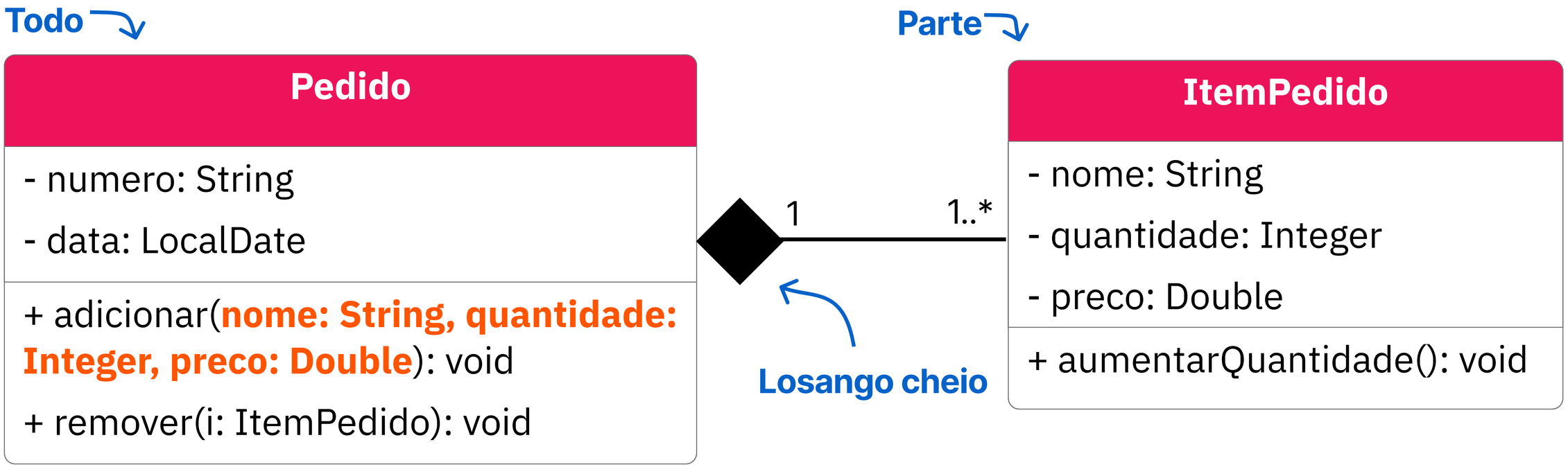
```java
public class Main {

  public static void main(String[] args) {
    Jogador jogador = new Jogador("Bob", 25);
    Time time = new Time("SPfut", "Polvo", true);
    time.adicionar(jogador);
  }
}
```

Objeto parte (Jogador)
existe Independente do
Objeto Todo (Time)

Composição

- Objeto parte só existe se o objeto todo existe.
- Objeto parte não pode ser parte de outro objeto.



Composição

Não há referência de ItemPedido fora do Pedido

```java
public class Main {

  public static void main(String[] args) {
    Pedido pedido = new Pedido("1", LocalDate.of(2026,10,10));
    pedido.adicionarItem("Bola", 1, 25.0);
  }
}
```

Criamos o ItemPedido dentro do Pedido

```java
public class Pedido {

  (...)

  public void adicionarItem(String nome, int quantidade,
        double preco) {
    ItemPedido item = new ItemPedido(nome, quantidade, preco);
    itens.add(item);
  }
}
```

Associação

```java
public class Professor {
    private String nome;

    public Professor(String nome) {
        this.nome = nome;
    }
}
```

```java
public class Aluno {
    private String nome;

    public Aluno(String nome) {
        this.nome = nome;
    }

    public void assistirAula(Professor professor) {
        System.out.println(nome +
        " está assistindo à aula de " + professor);
    }
}
```

Aluno se relaciona apenas em tempo de execução com Professor. **Aluno não tem atributo Professor.**

-

Obrigado!